Adding 3D Models to your scene

File Format: glTF / GLB



Fig. 10: The glTF logo.[1]

Website: https://www.khronos.org/gltf/

glTF (GL Transmission Format) is the standard file format for 3D models on the web. The Khronos Group, which created it, describes it as “The JPEG of 3D”. It comes in two variants: a JSON-based .gltf file, which often references separate texture and binary files, and .glb, which is a single, self-contained binary file bundling everything together. This course uses .glb files.[2]

Who’s behind it: glTF is developed and maintained by the Khronos Group. Version 1.0 was released in 2015 and version 2.0 in 2017.[3] In 2022, glTF 2.0 was ratified as the ISO/IEC 12113:2022 international standard.[4]

Remember: glTF is always +Y up and measured in meters. In our MindAR scene, 1 unit equals the width of the target image, so you scale the model to fit.

[1]Wikimedia Commons, GlTF logo.svg (n.d.). Public domain; Khronos Group trademark. Available at: https://commons.wikimedia.org/wiki/File:GlTF_logo.svg (Accessed: 13 October 2026).
[2]Khronos Group, glTF overview (n.d.).
[3]Wikipedia, glTF (n.d.). Available at: https://en.wikipedia.org/wiki/GlTF (Accessed: 13 October 2026).
[4]Khronos Group (2022).

The Khronos Group



Fig. 11: The Khronos Group logo.[5]

Website: https://www.khronos.org/

The Khronos Group is an industry consortium that writes the open standards your graphics card, browser, and XR headset use to talk to each other: for example OpenGL and Vulkan for graphics cards, WebGL for browsers, and OpenXR for XR devices.

Who’s behind it: The Khronos Group was founded in 2000,[6] and is a not-for-profit, member-funded industry consortium. It works similarly to the W3C, but focuses on graphics, 3D, and XR rather than on the web in general. Its goal is to continuously create open, royalty-free standards for graphics and related technology, including OpenGL, Vulkan, WebGL, OpenXR, and glTF.[7]

Remember: Khronos writes the standards, other companies build the software and hardware that implement them.

[5]Wikimedia Commons, Khronos Group logo.svg (n.d.). Public domain; Khronos Group trademark. Available at: https://commons.wikimedia.org/wiki/File:Khronos_Group_logo.svg (Accessed: 13 October 2026).
[6]Khronos Group (2026).
[7]Khronos Group, About (n.d.).

Around the corner: Interactable glbs



Interactable glbs (KHR_interactivity): a glTF extension that embeds behavior graphs, networks of nodes for events, logic, and animation, directly in the file. A model can then react without custom code for every platform, for example by playing an animation when the viewer approaches or by showing and hiding parts on input. Khronos submitted the extension for ratification on 16 July 2026. Babylon.js and the Khronos glTF Sample Viewer can already play such graphs.[8]

Get the example model and more Infos from Khronos here
Test it with this Babylon.js live demo here

Good To know

Babylon.js is a free, open-source 3D engine for the web, started at Microsoft in 2013 and written in TypeScript. Like Three.js (which A-Frame builds upon) it renders with WebGL and WebGPU and supports WebXR, but it is a more complete engine with built-in tools such as a scene inspector, physics, and a node material editor.

So, Three.js and Babylon.js are kind of competitors. Both are open-source JavaScript 3D engines for the web, and Babylon.js is a full engine with more built in, while Three.js is a lighter library that you assemble yourself.

Website: https://babylonjs.com/

[8]Khronos Group, glTF Interactivity Extension Submitted for Ratification (2026). Available at: https://www.khronos.org/news/press/gltf-interactivity-extension-submitted-for-ratification (Accessed: 13 October 2026).

Rendering in the Browser with WebGL



WebGL is the browser's connection to the GPU (Graphics Processing Unit). It's what lets a website draw 3D graphics using your device's graphics hardware, the same way the WebXR Device API lets a website talk to a VR/AR headset. A-Frame (and Three.js underneath it) uses WebGL to render everything you build in this course.[9]

The main conceptual difference between web based and native applications in regards to rendering is that there is an extra layer between the browser and your device's GPU on the web. A native app can talk to the GPU almost directly through the operating system, while a website has to go through the browser's rendering pipeline and JavaScript engine first. That used to make browser graphics much slower than app based, but modern GPUs and WebGL have made real-time 3D in the browser genuinely fast.

WebGL now, WebGPU tomorrow

WebGL is being succeeded by WebGPU, a newer graphics API built around how modern GPUs actually work today. [10] Just like WebXR, it is developed by the W3C and is currently, as of September 2026, a W3C Candidate Recommendation Draft.[11]

Why is WebGPU exiting? It gives us more computational budget to go bigger and busier! In simple scenes, the difference between WebGL and WebGPU rendering won't be apparent, but scenes with many objects or heavy GPU computing are able to run much faster with WebGPU.

Check out some WebGPU Demos:
three.js webgpu - ocean
https://webgpu-glass-drei.vercel.app/
False Earth
https://tympanus.net/Tutorials/GeometryPainterThreeJS/

[9]A-Frame, renderer (n.d.).
[10]Mozilla Corporation (n.d.).
[11]W3C, WebGPU, Candidate Recommendation Draft (2026). GPU for the Web Working Group. Available at: https://www.w3.org/TR/webgpu/ (Accessed: 13 October 2026).

From Blender to your AR scene

You can import and place the model in a web project of your choosing that you are working on currently. In this example, the model will be added to a MindAR scene with image tracking (Web AR).

1. Optimize and export a model in Blender
2. compress it
3. Add your model to the scene
4. measure performance

1. Optimize and export a model in Blender

Everything that makes a model heavy is decided here, before any compression. Compression can only shrink what you hand it. What drives file size and performance:

- **Vertices:** every vertex stores position, normal and UV coordinates, roughly 32 bytes. A mesh with 100,000 vertices is about 3 MB before compression. Keep only the vertices that are really needed.
- **Textures:** usually the biggest part of the file and of the GPU memory. A 2048×2048 texture takes 16 MB of GPU memory uncompressed.
- **Materials:** every material on a mesh is its own draw call. The same geometry with 20 materials renders slower than with 2.
- **Animations:** keyframes for every bone add up, and shape keys (morph targets) are heavy. Keep only the animation clips you need and keep the bone count low.

Think of it like having a performance budget for each scene and model in that scene that you can spend on these four traits (Vertex count, Texture count and size, material count, animation length and complexity). It is not possible to give you an exact target number for each of these traits, it is rather about distributing the performance budget among these traits to get the look you want at the lowest possible performance cost. In my personal practice experience however, I would start by merging materials and textures (in number and size) and see where I land before putting a lot of time into reducing vertex count and animation complexity.

I will not go into detail of merging materials and textures here, but you can wish for an input on this subject if it is something you want to learn more about. In a nutshell:

- Textures can be combined by setting up a new UV map for the model, mapping all areas that contain combinable textures onto one single texture. Then, the albedo color of the texture has to be baked from one UV map with the different textures, onto the single texture with the new UV map.
- When you have your combined texture, you can set up a single material that applies that texture to the model.

Checklist before export

- ☐ Sensible size: Check the dimensions in the N-panel (Item tab). For example MindAR, 1 unit is the width of your target image, so a model of about 1 m in Blender roughly fits the image width; fine-tune with the scale attribute.
- ☐ Apply transforms: select everything, Ctrl+A → All Transforms (scale 1, rotation 0).
- ☐ Set Origin to where the model should be ankered: For example, the origin is the point that sits on the center of your tracked image. Place the 3D cursor under the model, then Object → Set Origin → Origin to 3D Cursor.
- ☐ Remove what nobody sees: faces underneath and inside, hidden objects, cameras, lights, empties. Then File → Clean Up → Purge Unused Data.
- ☐ Reduce the mesh: Merge by Distance (M), a Decimate modifier for dense meshes such as scans, and bake fine detail into a normal map instead of keeping it as geometry. Target: under 50,000 triangles per model.
- ☐ Few materials: join objects that share a material (Ctrl+J). Aim for 1–3 materials per model.
- ☐ Principled BSDF only: it is the only shader that exports to glTF. Procedural textures and custom node setups must be baked into image textures first.
- ☐ Textures sized to the need: 1024×1024 for small objects, 2048×2048 only for hero objects seen up close. Use powers of two (512, 1024, 2048).
- ☐ Transparency: if a material has alpha, overlapping parts can render in the wrong order. Put a Math node set to Round between the texture's Alpha output and the Principled BSDF's Alpha input. Parts that really need soft transparency (glass, fades) go into their own object with Backface Culling on.
- ☐ Animations: delete unused Actions, keep the clip(s) you need.
- ☐ File name in lowercase, no spaces, no diacritics (ä,ö,...) e.g. lamp.glb. especially diacritics can cause trouble on web servers.

Export settings

(File → Export → glTF 2.0; labels differ slightly between Blender versions):[12]

- ☐ Format: glTF Binary (.glb)
- ☐ Include: Selected Objects only
- ☐ Transform: +Y Up
- ☐ Mesh: Apply Modifiers, UVs and Normals on
- ☐ Compression (Draco): off. We compress in the next step so that we can compare before and after.
- ☐ Animation: on only if the model is animated. Use the action filter if needed.

Before you continue, open the exported file in a glTF viewer to check that it looks right, and note its file size. gltf-viewer.donmccurdy.com or https://github.khronos.org/glTF-Validator/

---

[12]Blender Manual, glTF 2.0 (n.d.). Available at: https://docs.blender.org/manual/en/latest/addons/import_export/scene_gltf2.html (Accessed: 13 October 2026).

2. Compress the model

File size goal
there's no single correct number because it depends on your scene and device. But Google's own AR guidelines give a concrete reference point. Treat these as ceilings, not targets. a well-optimized model for this course should end up well under them:

- cap model size at 10MB ("Bigger models may result in poor user experience")
- recommended triangle count no higher than 100,000 (with 30,000–50,000 as "an ideal range")
- cap texture resolution at 2048×2048.[13]

Mesh Compression (Draco)
Fewer polygons means less work for the GPU on every single frame). Beyond starting with a simpler model, you can compress the geometry itself without visibly changing it using Draco, a geometry compression library made by Google.[14]

Textures Compression
Textures cost GPU memory and download time separately from the mesh. Two things matter here: resolution & file format
-> Use the smallest size that still looks good at the size the model will actually be displayed
-> WebP makes the download noticeably smaller than PNG or JPG at similar visual quality, but it does not reduce GPU memory

[13]Google, ARCore (n.d.).
[14]Google, Draco (n.d.).

Tool: gltf-transform

gltf-transform is a free, open-source command-line tool that handles mesh and texture compression together in a single command like this one:

```
gltf-transform optimize input.glb output.glb --compress draco
--texture-compress webp --texture-size 1024
```

This compresses the geometry with Draco, converts textures to WebP and limits them to 1024 px in one step[15]

Install and run
One time global installation. Run this in any terminal:

npm install --global @gltf-transform/cli

Then check the model before and after with which lists size, triangles, textures and draw calls:

gltf-transform inspect [your model name].glb

What happens inside Draco: it does not delete detail, it stores the vertex data in a much smaller form. Positions, normals and UVs are quantized (stored with fewer bits instead of 32-bit floats) and then entropy-coded, which typically makes the geometry 90–95% smaller. Your device unpacks it when the model loads, in a Web Worker so the page does not freeze.

Important: Draco makes the download smaller, it does not make rendering faster.
Frame rate depends on triangles, draw calls (number of materials) and textures.[16]

Texture formats Note: WebP makes the download smaller, but the GPU still stores every texture uncompressed, so the memory use from Step 2 stays the same. KTX2 (Basis Universal) is the only format that also shrinks GPU memory (roughly 4–8 times), but it needs an extra transcoder in the page and the KTX-Software tool installed. In this course we use WebP and reduce the resolution. KTX2 is the next step for bigger projects.

Compress your model with the command line at the beginning of this page.
Compare and note your numbers after compression.

Course target: 3–5 MB per model, textures 1024 px (2048 px at most).
If you are still above 5 MB, look at the biggest item first: gltf-transform inspect shows the size of every texture and mesh.

---

[15]glTF Transform, CLI docs (n.d.).
[16]A-Frame, gltf-model, Compression (n.d.). Available at: https://aframe.io/docs/1.7.0/components/gltf-model.html (Accessed: 13 October 2026).

Step 3: Add the model to your scene

This basic scene uses A-Frame and mindAR to create a web AR image tracking scene. It makes an <a-box> appear on top of a printed image. Remember, we use Vite to host it locally as https so we can test it on the phone.
Since you have already set this up as an exercise in a previous class, we will not discuss it here. If you missed the previous class, follow the setup guide from mindAR step by step and use the Vite setup guide from the previous basic A-scene cheat sheet to set up the hosting.

In index.html:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>basic-ar-scene</title>
    <script src="https://aframe.io/releases/1.5.0/aframe.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/mind-ar@1.2.5/dist/mindar-image-aframe.prod.js"></script>
  </head>
  <body>

    <a-scene
      mindar-image="imageTargetSrc: /targets.mind;"
      color-space="sRGB"
      renderer="colorManagement: true, physicallyCorrectLights"
      vr-mode-ui="enabled: false"
      device-orientation-permission-ui="enabled: false"
    >
      <a-camera position="0 0 0" look-controls="enabled: false"></a-camera>

      <a-entity mindar-image-target="targetIndex: 0">
        <a-box position="0 0 0.25" scale="0.5 0.5 0.5" color="#4CC3D9"></a-box>
      </a-entity>
    </a-scene>

  </body>
</html>
```

Make sure your compiled target image (targets.mind) is in the public/ folder of your project. Run *npm run dev* and test that the box actually appears on top of the image.

Remember: the image target is the origin of everything inside it. 1 unit is the width of the image, and the box is scaled to 0.5 and moved 0.25 forward (z), so it rests exactly on the image instead of sinking into it.

4. Replace the box with your model

Where to put the file
Vite serves everything in the public/ folder as-is, at the root of your site. Put the file in public/models/, for example public/models/lamp.glb, and it is available at /models/lamp.glb.[17]

Two ways to load a model in A-Frame
Directly:
gltf-model="url(/models/lamp.glb)" loads the model when the entity is created. Fine for a quick test, but the model pops in whenever it finishes loading.

With the A-Frame asset system:
declare the file once in <a-assets> (a child at the beginning of <a-scene>) and reference it by id. A-Frame waits until all assets are loaded before it starts the scene, which avoids pop-in for models you need right away.

```
<a-assets>
  <a-asset-item id="lamp" src="/models/lamp.glb"></a-asset-item>
</a-assets>
```

Put the <a-assets> block directly into the <a-scene>, before the camera. Then replace the <a-box> inside the image target with the model:

```
<a-entity mindar-image-target="targetIndex: 0">
  <a-entity gltf-model="#lamp" position="0 0 0" scale="0.5 0.5
0.5" rotation="90 0 0"></a-entity>
</a-entity>
```

The image target defines the coordinate system: the center of the image is the origin, 1 unit is the width of the image, and the z axis points out of the image toward you. A model with its origin at the bottom therefore stands on the image. glTF is +Y up, so if the image lies flat on a table, rotation="90 0 0" stands the model up. Use scale to fit the model to the image. If it floats or sinks, the origin is wrong (Step 2).

[17]Vite, Static Asset Handling: The public directory (n.d.). Available at: https://vite.dev/guide/assets#the-public-directory (Accessed: 13 October 2026).

5.Test the scene

Checking your work: once you've optimized your model, add the stats attribute to your scene to see if it actually helped: <a-scene stats>. This shows live performance metrics right in the browser, including fps and draw calls.[18]

The stats panel also reports Textures, Geometries and Vertices/Faces. To see the numbers and console messages on the phone itself, add Eruda, a small in-page developer console:[19]

```
<script src="https://cdn.jsdelivr.net/npm/eruda@3.4.1"></script>
<script>eruda.init()</script>
```

Alternatively connect the phone by USB.

Targets for this course (ceilings, not goals), measured on the phone:

- ☐ fps: a steady 60. Below 30 AR feels laggy.
- ☐ Calls (draw calls): under 300 for the whole scene (A-Frame best practices). Our rule of thumb for one model: under 10 (one per material and per separate mesh, so a joined model with 1–3 materials costs 1–3).
- ☐ Triangles: under 100,000 for the scene, ideally 30,000–50,000 (Google ARCore guideline).
- ☐ Textures: as few as possible, ideally one set of 2–3 images per model.
- ☐ File size: 3–5 MB per model, 10 MB for everything together (Google ARCore guideline).

Compare the original and the optimized model with these numbers. Compression changes the download size, not these values. Fewer triangles, fewer materials and smaller textures do.

---

[18]A-Frame, stats (n.d.).
[19]GitHub, liriliri/eruda repository (n.d.). Available at: https://github.com/liriliri/eruda (Accessed: 14 October 2026).

Exercise: Animated model

Add an optimized 3D model to your AR and VR scene that has an animation running in a loop. Use the course documents and A-Frame documentation.

The problem:
a-Frames gltf-model loads a glTF/glb file, but on its own it doesn't play any animation baked into it.

The solution:
Playing a looping animation needs one more component: The animation-mixer
It isn't part of A-Frame itself, but comes from a separate community library called aframe-extras[20]. Add its script tag after A-Frame's own, then check the aframe-extras documentation yourself for how to use the animation-mixer.

```
<script
src="https://cdn.jsdelivr.net/gh/c-frame/aframe-extras@7.7.0/dist/
aframe-extras.min.js"></script>
```

Where to find a model

- On Ilias there is a folder "image tracking example ressources" that contains the fbx model and target image of the Cyber Funk Studio Postcard. but feel free to go through this process with any model you want.
- You can use models from your own projects
- browse the Khronos Group's own glTF Sample Assets collection(filter by the "Animation" tag[21]) or search Sketchfab for a free, downloadable .glb model under a Creative Commons license[22]. Check that the model is rigged with an animation before downloading it.

[20]GitHub, c-frame/aframe-extras repository (n.d.).
[21]Khronos Group, glTF Sample Assets (n.d.).
[22]Sketchfab (n.d.).